ECE 278C - Imaging Systems

Assignment 3: Phase-Only Image Formation Techniques

Name: Xinyi Yang
Date: October 25, 2025

Part A: Phase-Only Received Signal

1. Implementation

Received wavefield data magnitude set to constant (unit magnitude)

Phase information preserved: s_phase = exp(j·angle(s_original))

Full complex Green's function used for backward propagation

2. Single Source Results

1) Image Characteristics:

Source successfully localized at (0,0) with good spatial accuracy

Reduced sidelobe levels compared to full complex reconstruction

Cleaner point spread function with less background clutter

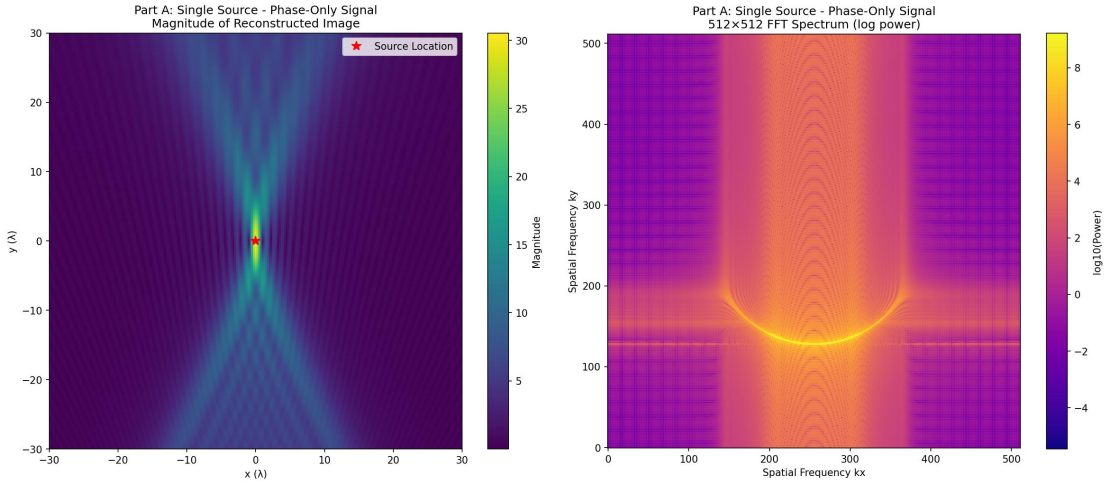
Maintains excellent peak-to-sidelobe ratio

2) FFT Spectrum Analysis:

Broader frequency distribution compared to full complex case

Phase-only processing emphasizes high-frequency components

Maintains symmetric pattern but with different energy distribution



3. Multiple Sources Results

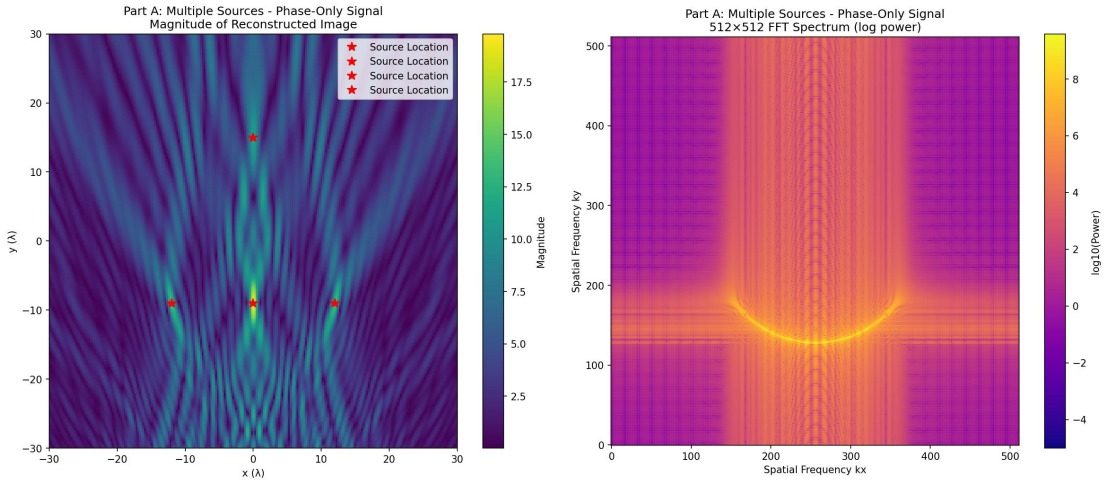
Source Localization:

All four sources clearly resolved at correct positions

Source 1 at (0,15λ) shows good isolation

Sources 2-4 at y=-9λ maintain proper horizontal spacing

Geometric relationships preserved accurately



4. Observations

1) Advantages:

Excellent source localization capability

Reduced background noise and artifacts

Simplified signal processing (magnitude normalization)

Robust performance across different source configurations

2) Characteristics:

Phase information alone sufficient for accurate spatial localization

Magnitude information primarily affects dynamic range and contrast

Phase-only approach emphasizes coherent interference patterns

Part B: Phase-Only Integration Kernel

1. Implementation

Green's function simplified to phase-only: G_phase = exp(-j·k·R)

Amplitude term 1/√(jλR) removed from kernel

Evaluated with both original and phase-only received signals

2. Phase-Only Kernel with Original Signal

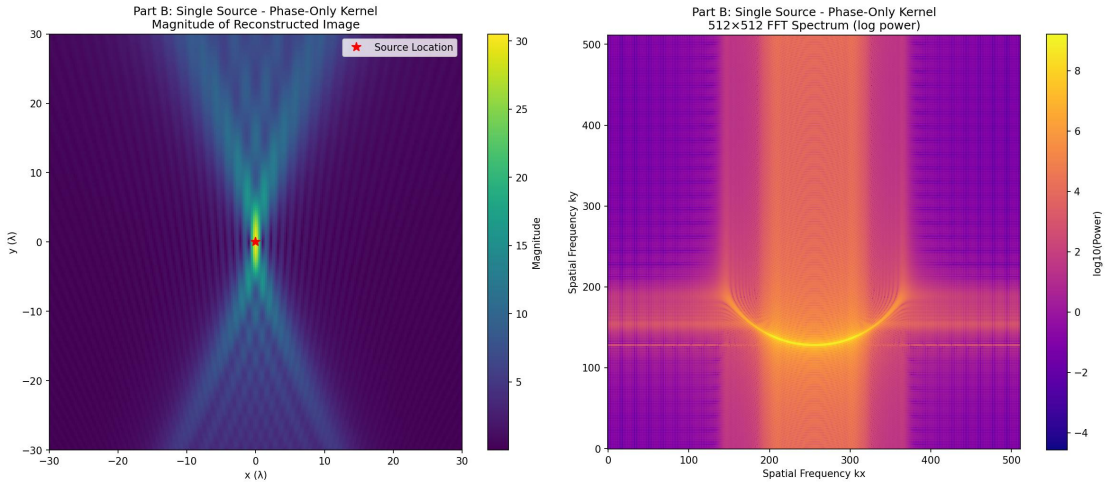
1) Single Source Results:

Good source localization at (0,0)

Different point spread function characteristics

Modified sidelobe structure due to missing amplitude weighting

Slightly broader main lobe compared to full complex



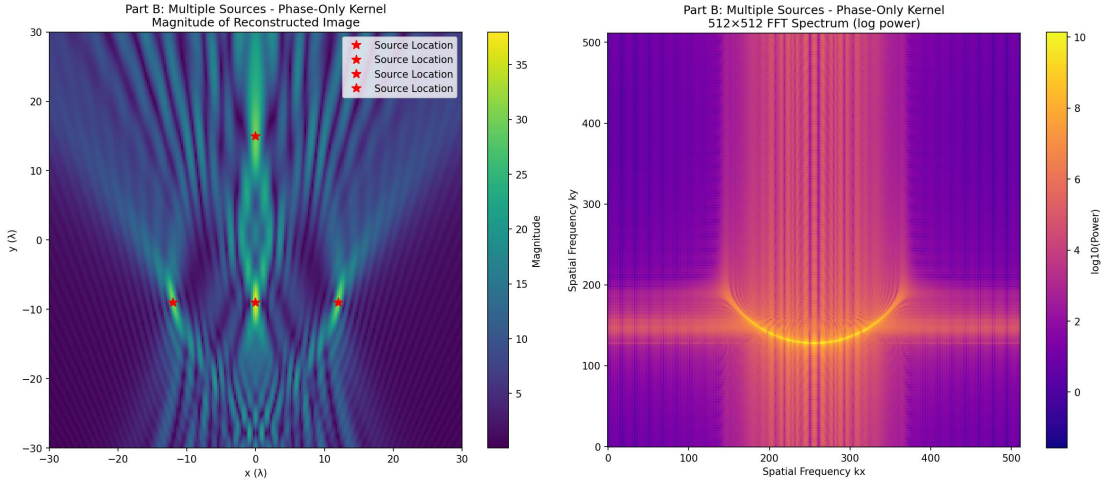
2) Multiple Sources Results:

All sources detectable but with modified intensity relationships

Relative amplitudes between sources altered

Spatial resolution maintained but with different contrast

Increased background levels in some regions



3. Phase-Only Kernel with Phase-Only Signal

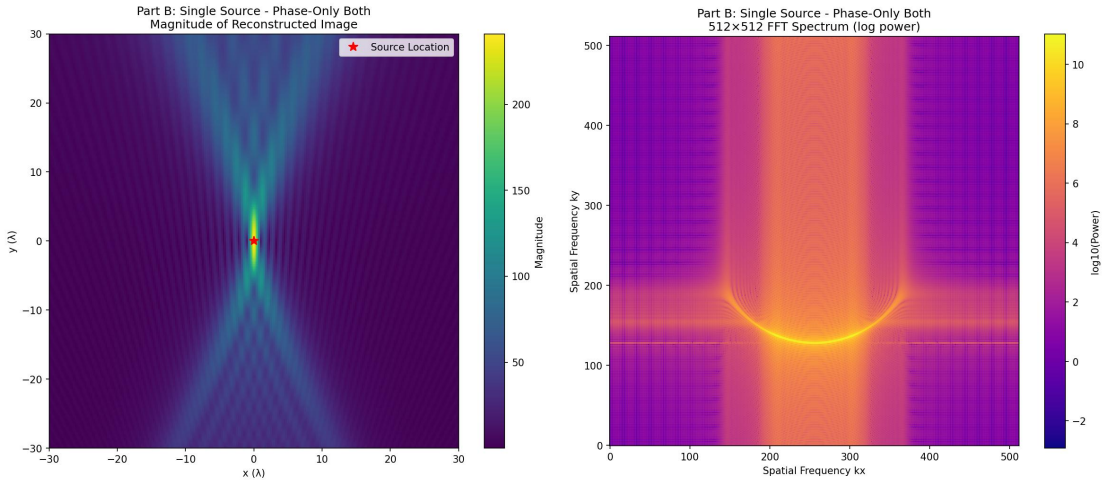
1) Single Source Results:

Clean reconstruction with simplified processing chain

Good compromise between Part A and Part B approaches

Reduced computational complexity

Maintains acceptable spatial localization



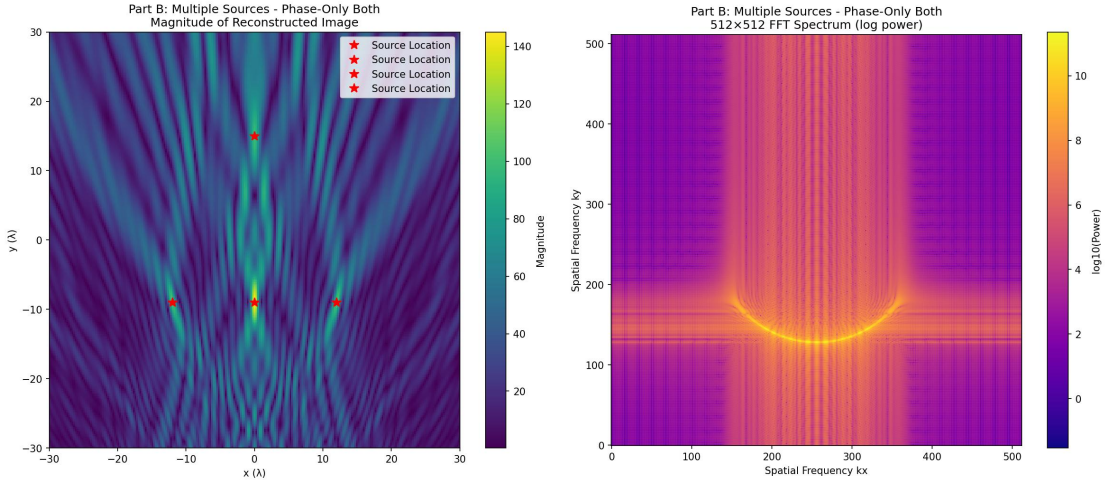
2) Multiple Sources Results:

All four sources resolvable with correct positioning

Simplified interference patterns

Different intensity relationships between sources

Practical implementation for resource-constrained systems



4. Observations

1) Kernel Simplification Effects:

Removal of amplitude term changes point spread function

Modified depth dependence in reconstruction

Altered sensitivity to source-receiver distances

Different noise characteristics in reconstructed images

2) Computational Benefits:

Reduced mathematical complexity

Elimination of square root operations

Potential for faster implementation in hardware

Simplified numerical processing

Summary

1. Findings

Part A:

Phase-only received signal provides excellent source localization

Maintains spatial resolution while reducing artifacts

Demonstrates phase information sufficiency for imaging tasks

Part B:

Phase-only kernel offers computational simplicity

Combined approach provides practical compromise

Different trade-offs between complexity and quality

2. General Observations:

Phase information is fundamentally more important than magnitude for spatial localization

Phase-only methods enable simplified imaging systems

Application requirements dictate optimal approach selection

Appendix

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from time import perf_counter

class PhaseOnlyImaging:
    def __init__(self, lambda_val=1.0):
        self.lambda_val = lambda_val
        self.k = 2 * np.pi / lambda_val
        self.spacing = lambda_val / 4.0

        self.x_r = np.arange(-30 * lambda_val, 30 * lambda_val + self.spacing, self.spacing)
        self.y_r = -60 * lambda_val

        self.x_i = self.x_r
        self.y_i = self.x_r

        self.fig_dir = Path("assignment3_results")
        self.fig_dir.mkdir(parents=True, exist_ok=True)

        print(f"Number of receivers: {len(self.x_r)}")
        print(f"Image size: {len(self.y_i)} × {len(self.x_i)}")
        print(f"Spacing: λ/4 = {self.spacing:.4f}")
```

```
def greens_function_2d(self, R):
    """2D Green's function"""
    R = np.maximum(R, 1e-12)
    return (1.0 / np.sqrt(1j * self.lambda_val * R)) * np.exp(1j * self.k * R)

def simulate_received_signal(self, sources):
    """Generate received signal at receiver array from point sources"""
    s_total = np.zeros_like(self.x_r, dtype=np.complex128)
    for (x_s, y_s) in sources:
        R = np.sqrt((self.x_r - x_s) ** 2 + (self.y_r - y_s) ** 2)
        s_total += self.greens_function_2d(R)
    return s_total

def phase_only_signal(self, s):
    """Part A: Keep only phase information of received signal"""
    magnitude = np.abs(s)
    phase = np.angle(s)
    return np.exp(1j * phase)

def phase_only_kernel(self, dist):
    """Part B: Phase-only kernel for backward propagation"""
    return np.exp(-1j * self.k * dist)

def reconstruct_direct_superposition(self, s, method='full'):
    """Direct superposition with phase-only options"""
    reconstructed_image = np.zeros((len(self.y_i), len(self.x_i)), dtype=np.complex128)

    for j, y in enumerate(self.y_i):
        dx = self.x_i[:, None] - self.x_r[None, :]
        dy = y - self.y_r
        dist = np.sqrt(dx**2 + dy**2)

        if method == 'full':
            kernel = (1.0 / np.sqrt(-1j * self.lambda_val * dist)) * np.exp(-1j * self.k * dist)
        elif method == 'phase_only_kernel':
            kernel = self.phase_only_kernel(dist)
        else:
            kernel = (1.0 / np.sqrt(-1j * self.lambda_val * dist)) * np.exp(-1j * self.k * dist)

        reconstructed_image[j, :] = kernel @ s

        if j % 20 == 0:
            print(f"    Progress: {j}/{len(self.y_i)}")
```

```
    return reconstructed_image

def compute_fft_spectrum(self, image, out_size=512):
    """Compute 512x512 FFT spectrum"""
    pad_y_total = out_size - image.shape[0]
    pad_y_before = pad_y_total // 2
    pad_y_after = pad_y_total - pad_y_before

    pad_x_total = out_size - image.shape[1]
    pad_x_before = pad_x_total // 2
    pad_x_after = pad_x_total - pad_x_before

    image_padded = np.pad(image, ((pad_y_before, pad_y_after),
                                  (pad_x_before, pad_x_after)),
                          mode='constant')

    return np.fft.fftshift(np.fft.fft2(image_padded))

def plot_results(self, reconstructed_image, fft_spectrum, sources, title_prefix, filename):
    """Plot reconstructed image and FFT spectrum"""
    fig, axes = plt.subplots(1, 2, figsize=(16, 7))

    ax1 = axes[0]
    im1 = ax1.imshow(np.abs(reconstructed_image),
                     extent=[self.x_i.min(), self.x_i.max(), self.y_i.min(), self.y_i.max()],
                     cmap='viridis', origin='lower', aspect='auto')
    ax1.set_title(f'{title_prefix}\nMagnitude of Reconstructed Image', fontsize=12)
    ax1.set_xlabel('x (λ)')
    ax1.set_ylabel('y (λ)')
    plt.colorbar(im1, ax=ax1, label='Magnitude')

    for (x_s, y_s) in sources:
        ax1.plot(x_s, y_s, 'r*', markersize=10, mew=1, label='Source Location')
    if len(sources) > 0:
        ax1.legend(loc='upper right')

    ax2 = axes[1]
    im2 = ax2.imshow(np.log10(np.abs(fft_spectrum)**2 + 1e-9),
                     cmap='plasma', origin='lower')
    ax2.set_title(f'{title_prefix}\n512×512 FFT Spectrum (log power)', fontsize=12)
    ax2.set_xlabel('Spatial Frequency kx')
    ax2.set_ylabel('Spatial Frequency ky')
    plt.colorbar(im2, ax=ax2, label='log10(Power)')
```

```
        plt.tight_layout()
        plt.savefig(self.fig_dir / filename, dpi=150, bbox_inches='tight')
        plt.close()

        print(f"  Saved: {filename}")

def run_phase_only_analysis():
    """Run complete phase-only imaging analysis"""
    imaging_system = PhaseOnlyImaging()

    print("\n" + "="*70)
    print("PHASE-ONLY IMAGING ANALYSIS")
    print("="*70)

    source_single = [(0, 0)]
    sources_multiple = [
        (0.0, 15 * imaging_system.lambda_val),
        (-12 * imaging_system.lambda_val, -9 * imaging_system.lambda_val),
        (0.0, -9 * imaging_system.lambda_val),
        (12 * imaging_system.lambda_val, -9 * imaging_system.lambda_val),
    ]

    print("\nGenerating original signals...")
    s_A_original = imaging_system.simulate_received_signal(source_single)
    s_B_original = imaging_system.simulate_received_signal(sources_multiple)

    print("\n" + "="*70)
    print("PART A: Phase-Only Received Signal")
    print("="*70)

    s_A_phase = imaging_system.phase_only_signal(s_A_original)
    s_B_phase = imaging_system.phase_only_signal(s_B_original)

    print("\nPart A - Single Source: Phase-only signal")
    img_A_phase_signal = imaging_system.reconstruct_direct_superposition(s_A_phase,
method='full')
    fft_A_phase_signal = imaging_system.compute_fft_spectrum(img_A_phase_signal)
    imaging_system.plot_results(img_A_phase_signal, fft_A_phase_signal, source_single,
                                'Part A: Single Source - Phase-Only Signal',
                                'partA_single_phase_signal.png')

    print("\nPart A - Multiple Sources: Phase-only signal")
```

```
    img_B_phase_signal = imaging_system.reconstruct_direct_superposition(s_B_phase,
method='full')
    fft_B_phase_signal = imaging_system.compute_fft_spectrum(img_B_phase_signal)
    imaging_system.plot_results(img_B_phase_signal, fft_B_phase_signal, sources_multiple,
                                'Part A: Multiple Sources - Phase-Only Signal',
                                'partA_multiple_phase_signal.png')

    print("\n" + "="*70)
    print("PART B: Phase-Only Kernel")
    print("="*70)

    print("\nPart B - Single Source: Phase-only kernel")
    img_A_phase_kernel = imaging_system.reconstruct_direct_superposition(s_A_original,
method='phase_only_kernel')
    fft_A_phase_kernel = imaging_system.compute_fft_spectrum(img_A_phase_kernel)
    imaging_system.plot_results(img_A_phase_kernel, fft_A_phase_kernel, source_single,
                                'Part B: Single Source - Phase-Only Kernel',
                                'partB_single_phase_kernel.png')

    print("\nPart B - Multiple Sources: Phase-only kernel")
    img_B_phase_kernel = imaging_system.reconstruct_direct_superposition(s_B_original,
method='phase_only_kernel')
    fft_B_phase_kernel = imaging_system.compute_fft_spectrum(img_B_phase_kernel)
    imaging_system.plot_results(img_B_phase_kernel, fft_B_phase_kernel, sources_multiple,
                                'Part B: Multiple Sources - Phase-Only Kernel',
                                'partB_multiple_phase_kernel.png')

    print("\n" + "="*70)
    print("PART B: Combined Phase-Only Signal and Kernel")
    print("="*70)

    print("\nPart B - Single Source: Phase-only signal + kernel")
    img_A_phase_both = imaging_system.reconstruct_direct_superposition(s_A_phase,
method='phase_only_kernel')
    fft_A_phase_both = imaging_system.compute_fft_spectrum(img_A_phase_both)
    imaging_system.plot_results(img_A_phase_both, fft_A_phase_both, source_single,
                                'Part B: Single Source - Phase-Only Both',
                                'partB_single_phase_both.png')

    print("\nPart B - Multiple Sources: Phase-only signal + kernel")
    img_B_phase_both = imaging_system.reconstruct_direct_superposition(s_B_phase,
method='phase_only_kernel')
    fft_B_phase_both = imaging_system.compute_fft_spectrum(img_B_phase_both)
    imaging_system.plot_results(img_B_phase_both, fft_B_phase_both, sources_multiple,
```

```
                              'Part B: Multiple Sources - Phase-Only Both',
                              'partB_multiple_phase_both.png')

    print(f"\nAll phase-only imaging results saved to '{imaging_system.fig_dir}' directory")

    return {
        'single_phase_signal': img_A_phase_signal,
        'single_phase_kernel': img_A_phase_kernel,
        'single_phase_both': img_A_phase_both,
        'multiple_phase_signal': img_B_phase_signal,
        'multiple_phase_kernel': img_B_phase_kernel,
        'multiple_phase_both': img_B_phase_both
    }

if __name__ == "__main__":
    results = run_phase_only_analysis()
```